

Embedded Systems II

EEE3096/5S



FINAL EXAM

8 December 2020

3 hours

*Examination Prepared by:
Simon Winberg and Jane Wyngaard*

Last Modified: 24-Nov-2020

REGULATIONS

Dear Students,

This is an open-book exam. You may use the course notes and textbook, however you may not use additional resources such as websites. You need to abide by the honor declaration that you will not cheat or assist others to cheat on this examination.

If you are caught cheating, you will be referred to University Court for disciplinary procedures.

Make sure that you **put** your **student name and student number** clearly in your submissions for this assignment. Clearly indicate the course code **EEE3096S or EEE3095S** and a title **Final Exam** on your answer sheet(s).

DO NOT TURN OVER UNTIL YOU ARE TOLD TO

Exam Structure

Marked out of 100 marks. 180 minutes.

RULES

- NB. 
- You must write your name and student number on each answer book.
 - Write the question numbers attempted on the cover of each book.
 - Start **each section on a new page**.
 - Make sure that you cross out material you do not want marked. Your first attempt at any question will be marked if two answers are found.
 - Use a part of your script to plan the facts for your written replies to questions, so that you produce carefully constructed responses.
 - Answer all questions, and note that the time for each question relates to the marks allocated (which implies there are a few minutes to allow you to think and shuffle papers etc).



Structure of exam:

Multiple Choice ☐ **Short Answers** ☐ **Design Questions to test your mettle**

NOTE: Don't leave out the section because it counts the most!

SECTION A:

MULTIPLE CHOICE AND SOME EASYPEASY QUESTIONS [10 x 4 = 40 marks]

Question A.1. [4 marks]

In SPI, when moving from idle to start, which is controlled by the master, which of the following needs to occur before the slave will start receiving the data so that the master can start sending the data?

- a) The Master Input Slave Output line must be held LOW
- b) The slave select line must change from HIGH and to LOW
- c) The slave select line must change from LOW to HIGH
- d) The Master Output Slave Input line must be held HIGH

Question A.2. [4 marks]

Which of the following is a list of different memory technologies used in a modern computer, listed in order of increasing Read/Write speeds

- a) Flash based CPU Registers, DDR DRAM memory, Solid state drive
- b) Flip-flop based SRAM, Capacitor based DRAM, NOR/NAND based Flash
- c) Capacitor based SRAM, flip-flop based DRAM, Flash based SSD
- d) Flip-flop based DRAM, Capacitor based SRAM, Single Transistor based Flash

Question A.3. [4 marks]

Multi-issue pipelined ALUs are standard in modern CPU designs. The following is an accurate description of how multi-issue and pipelining increase performance.

- a) Pipelining decreases the CPI of a CPU while multi-issue capacity increases a CPU's IPC, both of which improve CPU performance.
- b) Pipelining increases the CPI of a CPU while multi-issue capacity decreases a CPU's IPC, both of which improve CPU performance.
- c) Pipelining involves splitting the execution of an instruction into multiple stages and allowing stages to overlap thereby increasing a CPU's throughput. Multi-issue architectures have multiple CPU cores on single die enabling multiple processes to execute in parallel.
- d) Both pipelining and multi-issue architectures exploit the abundance of transistors made available by modern fabrication techniques to increase the clock speed at which a CPU can be run.

Question A.4. [4 marks]

When selecting a microcontroller, a decision must be made regarding which ISA is most appropriate. When considering the available ARM ISAs, the following would be relevant considerations:

- a) If the microcontroller is multi-issue, how deeply pipelined it is, and if it supports an RTOS.
- b) How large the cache is, if it supports Linux or another operating system, how much power it consumes.
- c) What IO is supported, cost, available PCBs or dev kits.
- d) Width of registers, inclusion of NEON instructions, the memory model, and what privilege levels are specified.

Question A.5. [4 marks]

In CPU pipelined to execute in stages (Fetch, Decode, Execute, Memory Access, Writeback), various potential hazards must be handled by the hardware. Executing the following assembly instructions will cause which type of hazard.

MUL R1, R2, R3

SUB R6, R1, R8

- a) Data Hazard
- b) Structural Hazard
- c) Control Hazard
- d) Stall Hazard

Question A.6. [4 marks]

In an operating system (OS), preemptive multitasking is used to schedule the execution of processes, which of the following is a valid description of preemptive multitasking.

- a) To ensure optimal execution efficiency, the OS kernel runs continuously on 1 CPU core, from where it manages execution of processes on all other available cores using a round robin scheduling algorithm.
- b) Multiple processes are assigned to execute on the CPU for short periods, process execution is interrupted by either a hardware or periodic interrupt, after which the interrupt handler hands control back to the OS to select the next process to run.
- c) The kernel provides an interface between userspace applications and hardware drivers, the scheduler application running in user space dynamically rotates access to these hardware resources based on the nice number assigned to each
- d) None of above.

Question A.7. [4 marks]

Below is C code describing a function that utilizes bitwise operations. What it is doing is assigning only those bits of the parameter value, corresponding to mark, to the byte at address data[index]. Select the option below indicating what assembly code would effectively implement this function. Assume that dataadr is stored close to the writemark function (i.e. it stores the 32-bit value address of the data array that is located elsewhere in memory). You can assume that 'call before store' is used for involving the function, so no worry with stacks ...

LINE	CODE
01	<code>extern unsigned char data;</code>
02	
03	<code>void writemask (int index, unsigned char value, unsigned char mask)</code>
04	<code>{</code>
05	<code> unsigned char temp;</code>
06	<code> temp = data[index];</code>
07	<code> temp = temp (value & mask);</code>
08	<code> data[index] = temp;</code>
09	<code>}</code>

Select the correct answer below:

a)

LINE	CODE
01	<code>writemask:</code>
02	<code> ldr r4, dataadr</code>
03	<code> ldrb r3, [r4, r0] @ read a byte</code>
04	<code> and r1, r1, r2 OR r3</code>
05	<code> strb r1, [r4, r0]</code>
06	<code> bx lr</code>
07	<code>dataadr: .word data @ address of data</code>

b)

LINE	CODE
01	<code>writemask:</code>
02	<code> ldr r4, dataadr</code>
03	<code> ldrb r3, [r4]</code>
04	<code> and r1, r1, r3</code>
05	<code> orr r0, r3, r1</code>
06	<code> strb r3, [r4, r0]</code>
07	<code> bx lr</code>
08	<code>dataadr: .word data @ address of data</code>

c)

LINE	CODE
01	<code>writemask:</code>
02	<code> ldr r4, dataadr</code>
03	<code> ldrb r3, [r4, r0] @ read a byte</code>
04	<code> and r1, r1, r2</code>
05	<code> orr r3, r3, r1</code>
06	<code> strb r3, [r4, r0]</code>
07	<code> bx lr</code>
08	<code>dataadr: .word data @ address of data</code>

d)

LINE	CODE
01	<code>writemask:</code>
02	<code> ldr r4, dataadr</code>
03	<code> adds r4, r4, r0</code>
04	<code> ldrb r3, [r4] @ read a byte</code>
05	<code> and r1, r1, r2</code>
06	<code> orr r1, r3, r1</code>
07	<code> cmp r1, #0</code>
08	<code> strbne r3, [r4]</code>
09	<code> bx lr</code>
10	<code>dataadr: .word data @ address of data</code>

Question A.8. [4 marks]

Computer engineers and computer scientists use lots of acronyms. They should probably use less when interacting with normal society. Indeed, some probably used textspeak type language before it became fashionable ... not that it helped make them cooler back then. So on that theme, for this question you need to connect the acronyms on the left with the correct meaning on the right. The are four acronyms to explain in the order of the acronyms on the left indicate the number of the row on the right that it maps to, you simply need to list four numbers in the right order (e.g. if they're in the right order already, the answer is 1,2,3,4).

Acronym	→	#	Explanation
SBC		1	System On Chip
GPIO		2	Generic Processor Integration and Organization [model]
SOC		3	Processor Linked Binary
PLB		4	Set Boot Core [i.e., to select core that boots the boot loader]
		5	Secondary Output Controller
		6	General Purpose Input Output
		7	Single Board Computer
		8	Programmable Logic Block

Question A.9. [4 marks]

Consider that we want to count the number of bits that are set in the register x. The starting point to the module has been provided below. You need to select which option below is the right answer for which of these code options you can replace the '*put code here*' line to achieve the objective.

```
module numbits ( x, n );  
    // define the ports  
    input [3:0] x;  
    output [1:0] n;  
    // define the registers  
    reg n;  
  
    // put code here!  
  
endmodule
```

Select the right code option below:

a)

```
always @(x)
begin
    n = 0;
    if (x&1) n = n + 1;
    else if (x&2) n = n + 1;
    else if (x&4) n = n + 1;
    else if (x&8) n = n + 1;
end
```

b)

```
always @(x)
begin
    n = (x&1) + (x&2) + (x&4) + (x&8);
end
```

c)

```
always @(x)
begin
    n = {{x[2]|x[3]}, {x[0]|x[1]}};
end
```

d) None of the above.

Question A.10. [4 marks]

Let's consider the many computer-based systems that are out in the world. If we narrow down to considering particularly those that are classified as: real-time systems (RTS), real-time and embedded (RTES) and embedded systems (ES). Which is actually the biggest of these categories, bearing in mind obviously that a real-time system could be implemented using a general purpose computer platform. Select the best option below ...?

- a) Real-time, but non-embedded, is the biggest (by far!).
- b) Real-time and embedded is the biggest.
- c) Embedded is the biggest.
- d) Embedded but not real-time is the biggest.

SECTION B:

SHORT ANSWERS – NOT QUITE SO EASY QUESTIONS [30 marks]

NOTE to examiner: For 2020 it is recommended to make these a combination of Multiple-choice, true/false and short (e.g. <200 word) answers that can be put into a textbox (i.e. not needing drawings).

Question B.1. [15 marks]

The following questions relate to the below Verilog code. Have a look at the code and answer the questions that follow.

Verilog Code Snippet

```
/* Mystery method using exceedingly poorly named interface and variables
   and an almost total lack of comments. A good example of how not
   to develop readable code ... but serves a purpose for an exam question. */

module mymodule ( rst, clk, a, b, c);
  // Port Declaration
  input  rst, clk; // that's not bad names, the designer started off keen
  input  a;        // then he got sloppy.
  input  [7:0] b;
  output reg c;
  reg    [7:0] x;
  reg    started;

  always @ (rst)
    if (rst == 1'b1)
      begin
        c = 0;
        x = 0;
        started = 0;
      end

  always @ (posedge clk)
    begin
      if (started == 1'b0)
        begin
          if (a == 1'b1) started <= 1'b1;
        end
      else
        begin
          if (c == 1'b0)
            begin
              x <= x + 1;
              if (x == b) c <= 1'b1;
            end
          end
        end
    end
endmodule
```

questions on next page ...

Questions to answer for B1:

B1.1. Look at how the input a is being utilized and select which option below provides an effective description of what a is used for (select only one option below):

- a) The 'a' input is some sort of start trigger, when it is high it sets the 'started' flag that starts the system counting up using the x register.
- b) The 'a' input tells the module what value to count up to, in particular the value that 'x' needs to count up to (from whichever value it was at previously).
- c) The 'a' input actually does two things, it operates as an 'active' line, that is to say that 'x' will only count up if 'a' is high, but it also acts as a start trigger that engages the counting up of 'x'.
- d) The 'a' input works together with the 'c' output: while 'a' is high, 'x' will increment and each time this happens 'c' will be set to high to confirm an increment occurred.

[3 marks]

B1.2. Is the following **true** or **false**? (Explain your answer.)

The reset (i.e. rst line) of mymodule is a clock synchronous reset.

[3 marks]

B1.3. Is the following **true** or **false**? (Explain your answer.)

If input b = 0, then mymodule module will never set c to 1.

[3 marks]

Consider the following testbench code, in relation to the code for mymodule, for both questions B1.4 and B1.5...

```
// Code to test mymodule
module test_mymodule ();
    // Register declarations
    reg rst, clk, a, c;
    reg [7:0] b;

    // instantiate the module under test
    mymodule muut ( rst, clk, a, b, c);

    // code for exercising the module
    initial
        begin
            // set initial values
            rst = 0; clk = 0; a = 0; b = 0;

            // regs to display when changed:
            $display("rst clk a  b  c  muut.x");
            $monitor("%b  %b  %b%d %b%d", rst, clk, a, b, c, muut.x);
            // note that muut.x above links to x within muut

            // start test...
```



```

rst = 1;
#10 clk = ~clk;

// pretty shoddy comments from here!
rst = 0; a = 0; b = 1;
#10 clk = ~clk;

a = 1;
repeat (5)
begin
    #10 clk = ~clk;
end
end
endmodule

```

B1.4. Considering the testbench code above, what would the output be when it is run?

a) <table><tr><td>rst</td><td>clk</td><td>a</td><td>b</td><td>c</td><td>muut.x</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>2</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>2</td></tr></table>	rst	clk	a	b	c	muut.x	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1	0	0	0	1	1	1	0	0	0	0	1	1	0	0	0	1	1	1	0	1	0	0	1	1	1	2	0	1	1	1	0	2	b) <table><tr><td>rst</td><td>clk</td><td>a</td><td>b</td><td>c</td><td>muut.x</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>2</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>2</td></tr></table>	rst	clk	a	b	c	muut.x	1	0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	1	0	1	0	0	1	1	0	1	0	1	1	1	0	2	0	0	1	1	1	2
rst	clk	a	b	c	muut.x																																																																																																								
1	0	0	0	0	0																																																																																																								
1	1	0	0	0	0																																																																																																								
0	0	1	1	0	0																																																																																																								
0	1	1	1	0	0																																																																																																								
0	0	1	1	0	0																																																																																																								
0	1	1	1	0	1																																																																																																								
0	0	1	1	1	2																																																																																																								
0	1	1	1	0	2																																																																																																								
rst	clk	a	b	c	muut.x																																																																																																								
1	0	0	0	0	0																																																																																																								
0	0	0	1	0	0																																																																																																								
0	1	0	1	0	0																																																																																																								
0	0	1	1	0	0																																																																																																								
0	1	1	1	0	1																																																																																																								
0	0	1	1	0	1																																																																																																								
0	1	1	1	0	2																																																																																																								
0	0	1	1	1	2																																																																																																								
c) <table><tr><td>rst</td><td>clk</td><td>a</td><td>b</td><td>c</td><td>muut.x</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>2</td></tr></table>	rst	clk	a	b	c	muut.x	1	0	0	0	0	0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1	1	0	0	0	0	1	1	0	0	0	1	1	1	0	1	0	0	1	1	0	1	0	1	1	1	1	2	d) <table><tr><td>rst</td><td>clk</td><td>a</td><td>b</td><td>c</td><td>muut.x</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>2</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>2</td></tr></table>	rst	clk	a	b	c	muut.x	1	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0	1	1	1	0	0	0	0	1	1	0	1	0	1	1	1	0	1	0	0	1	1	0	2	0	1	1	1	1	2
rst	clk	a	b	c	muut.x																																																																																																								
1	0	0	0	0	0																																																																																																								
0	1	0	1	0	0																																																																																																								
0	0	1	1	0	0																																																																																																								
0	1	1	1	0	0																																																																																																								
0	0	1	1	0	0																																																																																																								
0	1	1	1	0	1																																																																																																								
0	0	1	1	0	1																																																																																																								
0	1	1	1	1	2																																																																																																								
rst	clk	a	b	c	muut.x																																																																																																								
1	0	0	0	0	0																																																																																																								
1	1	0	0	0	0																																																																																																								
0	0	0	1	0	0																																																																																																								
0	1	1	1	0	0																																																																																																								
0	0	1	1	0	1																																																																																																								
0	1	1	1	0	1																																																																																																								
0	0	1	1	0	2																																																																																																								
0	1	1	1	1	2																																																																																																								

[3 marks]

B1.5. Consider that we changed the line:

```
always @ (posedge clk)
```

in mymodule to become:

```
always @ (posedge clk or negedge clk)
```

What effect would this have when you try to run the same implementation of test_mymodule on this revised version of mymodule? Choose the answer that best fits below:

- a) It will now take just 5 changes of clk for c to change to 1.
- b) It will now take just 6 changes of clk for c to change to 1.
- c) The muut.x value will change to 2 in the transition of clk after b has become 1.
- d) This solution will still take the same number of transitions of clk for x to change to 1, however the change of a from 0 to 1 will happen earlier.

Question B.2. [15 marks]

The following code snippet relates to questions B.2.1 to B.2.3.

```
#include <stdio.h>
2
3 #define red "\x1B[1;31m" //ANSI Colour Code for red
4 #define blue "\x1B[1;36m" //ANSI Colour Code for cyan
5
6 void print_red(char* s){printf("%s %s %s \n",red,s,red);}
7 void print_blue(char* s){printf("%s %s %s \n",blue,s,blue);}
8
9 void my_printer(void (*colorprinter)(char*), char* s){
10     colorprinter(s);
11 }
12
13 int main(){
14     char s1[10] = "EEE3096S";
15     char s2[10] = "2020";
16
17     my_printer(&print_blue,s1);
18     my_printer(&print_red,s2);
19
20 #ifdef DEBUG
21     printf("This is a debug message in %s at %d\n",__FILE__, __LINE__);
22 #endif
23     return 0;
24 }
```

Code listing B.2 in file main.c

B2.1

Name 3 actions that the c preprocessor cpp will carry out when gcc is called on the code in listing B.2?

[3 marks]

B2.2

What do the flags in the following command do when gcc is used to compile the code in listing B.2?

\$ gcc -g -D DEBUG -Wall -o colourprinter main.c

[2 marks]

B2.3

If compiled as in B2.2 what will the output of the code in listing B.2 be when run ?

[3 marks]

B2.4

Part of the reason Linux is so widely used in embedded systems is because it's open source and therefore free. Give 3 other examples of how the Linux kernel is highly suitable for use in embedded systems.

[3 marks]

B2.5

Cache hierarchies improve application performance and make it appear to an application (up until a point) that it has access to infinite high speed memory. This is possible due to the principles of spatial and temporal locality. Define both and describe how cache hierarchies and memory intentional management achieves the above.

[4 marks]

SECTION C:
TESTING YOUR METTLE QUESTIONS [30 marks]

Note: All questions in this section are compulsory. This section provides an additional opportunity to pass ELO5 Aspect B.1: Understanding design of ES as a complex process.

C.1

In operating systems, explain the difference between user and kernel space, how they interact, and give examples of what types of processes are run in each.

[4 marks]

C.2

In the context of computer architecture, operating systems, and writing applications, explain the difference between a thread and a process.

[5 marks]

C.3

From ISA, through CPU architecture, to application design, computer scientists have found many ways of exploiting parallelism in a multi-layered manner that all increase program performance in widely different manners. Describe 2 types of parallelism (hardware or software or both) used in a modern computer to increase program performance.

[5 marks]

(exam continues on next page ...)

The Following Scenario Relates to Questions C.4 - C.7 that follow

Take some time to read through this aspect carefully; the time for the exam is planned on you taking about 15 minutes reading and making sense of the scenario.

Scenario: The Nature Sense IoT Product

Consider that you are involved in developing a system, called the NatureSens, that connects to various sensor and output devices that are further explained below. The figure below shows a conceptual illustration of the system.

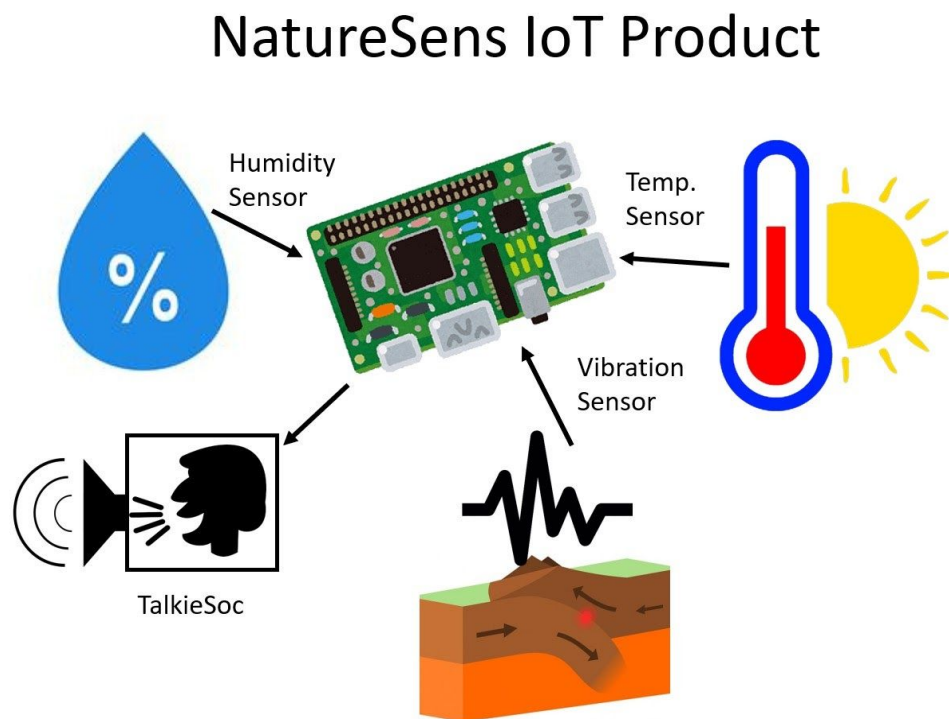


Fig. 1: Concept drawing of NatureSens IoT Product scenario used in this section

Operation of NatureSens: The system connects via I2C to two slow input devices, a temperature sensor and a humidity sensor, and to two faster SPI devices, an ADC that operates at high speed to measure vibrations and potential earth tremors from an accelerometer sensor, and the TalkieSoc that provides user output.

The TalkieSoc converts text to speech and has a full-duplex SPI interface. The microprocessor of the platform is the master in the system coordinating the sensing and actuating of these various devices. The NatureSens connects to a wireless network but you don't need to worry much with that aspect besides knowing that the device sends out data measurements as per the required operation below.

The TalkieSoc device (which connects to a speaker) will announce messages (e.g. for people that are too absorbed in their computer work during lockdown to realize what is going on around them). The TalkieSoc will alert the user to environmental conditions, e.g. "Dear Joe, it seems to be getting rather hot in here, maybe take a break and open a window." Or "Watch out Joe! Sounds like an Earthquake on the way!".

The NatureSens has three modes as described below, and it starts in 'quiet' mode. Note that the operations in 'always' must always be accommodated as well as possible (but when it is busy with important tasks such as sampling or running `calc_richter` the handling of 'always' can be done when time permits).

- Always operations
 - The temperature sensor measurement and humidity sensor measure must be read every minute (not necessarily at the same time). And logged to memory (i.e. **log_tehu()**).
 - Read a 5s block of accelerometer samples from the ADC every 10s; this is started by a call to **samples_read()** and will be carried out as a background process. The function **samples_avail()** returns 1 when the samples have been read and 0 otherwise.
 - Once the samples are read, the function **calc_richter()** is called, this is a blocking call and cannot be backgrounded (it takes 200ms to complete but you don't need to worry about that, you can also assume call_richter will automatically add the reading to the log).
- Quiet Mode
 - The latest log must be reported to the cloud every 1h (this is done by **synchlog()**), note that when synchlog is called it will do the synch in the background not disturbing other tasks.
 - If temperature is above 30 deg C it must annunciate **Talkie.send("It is hot in here!")** every 10 minutes (i.e., to encourage the inhabitants to improve the ventilation).
 - If the Richter scale measure (the current value in variable **richter**) is above 2 then mode changes to 'alert' mode.
- Alert mode
 - In this mode, the announcement **Talkie.send("Quake Alert")** must be done at the start of the state and every 5 minutes after that.
 - The latest log must be reported to the cloud every 5 min (also by **synchlog()**) in this mode.
 - Once the Richter scale measure has been below 2 for more than 20 minutes then the system returns to quiet mode.

Answer the following questions ...

C.4

Consider the above description of the NatureSens system. From the system classifications options below, choose which is the most accurate classification for this system and give a short motivation for your answer.

Options are:

- a) Embedded Systems (ES)
- b) Real Time Systems (RTS)
- c) Real Time Embedded Systems (RTES)
- d) None of the above

Motivation for answer: _____ (select that option in setting up the question)

[2 marks]

C.5

The NatureSens system has not been defined as a safety-critical system by its designers despite it sounding like it is doing important measurements. Briefly motivate why the designers have decided not to classify it as safety-critical. Bring in to your motivation what soft and hard deadlines are and if this system has either or both of these deadlines.

[3 marks]

C.6

Consider the starting point state machine diagram given below. There are two main categories of state machines, namely Moore machines and Mealy machines. Answer the following:

- a) Firstly, is the figure below a Moore machine or a Mealy machine?

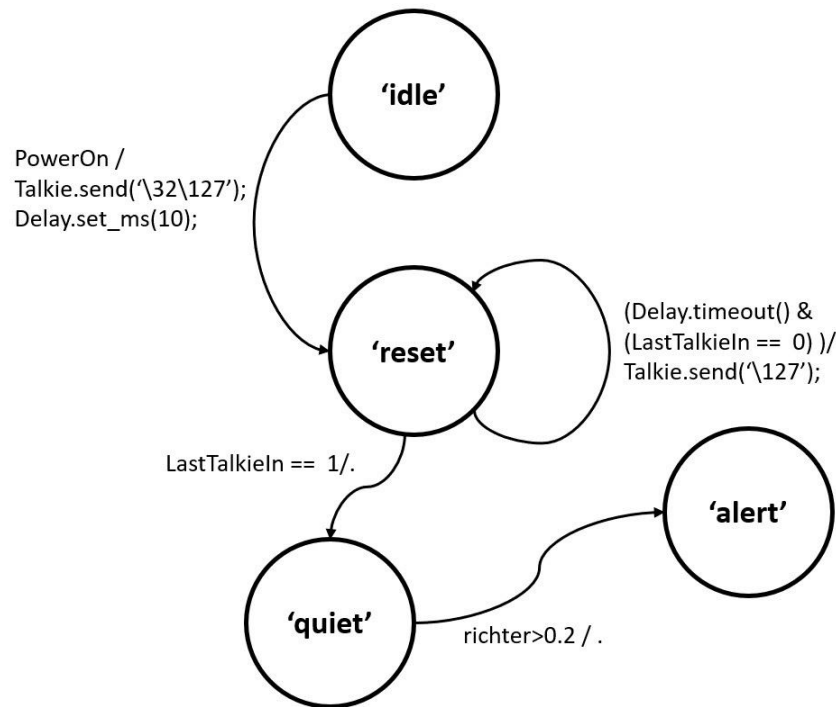
[1 mark]

- b) What would be a possible advantage of using a UML statechart instead of this type of diagram? Explain a benefit of statecharts that has merit to use in consideration of this application.

[2 marks]

- c) Attempt to complete the **'quiet' mode** of operation in this state machine (note you only need to show the 'quiet' mode, the modeling of any other modes is *not* needed). You can make any additions or changes to the existing incoming or outgoing links to/from 'quiet' mode state(s) e.g. to make your implementation earlier or to more things more correct. Note you may add further states or to totally reimplement what state is activated after the 'reset' state.

[5 marks]



Page over for next question

C.7

In the NatureSens system one of the designers implemented a bitbanging solution to communicate with the temperature and humidity sensors (see code below). The team leader who has had years of experience with I2C was shocked and horrified by the solution - mainly in terms of efficiency and accuracy considerations. The implementation was working (at least in the simple cases tested) so it is not a problem in terms of the sequencing changes in the SDA and SCL lines. Suggest some ideas on what may be wrong with this code and how it could be improved, you can make suggestions on how existing libraries or built-in I2C controllers (which is already available on the NatureSens platform) could be utilized and how that may improve the quality and scalability of the code solution for future upgrades of the product.

[3 marks]

```
/** Project: NatureSens - I2C Bitbang Write Module */

// these are defined in another module
void set_I2C_SDA_hi();
void set_I2C_SDA_lo();
void set_I2C_SCL_hi();
void set_I2C_SCL_lo();

/** do a busy delay to force the processor to delay for us microseconds */
int precise_delay ( int us )
{
    // the annoying optimizer messes with the code hence my funny
    // calculation in the inner loop to make sufficient delay
    // (return value is ignored by caller)
    int i, j;
    int ret = 0;
    for (i = 0; i < us; i++) { // takes 8ns for increment and termination testing
        for (j=0; j<25; j++)
            ret += (j+us)&i; // each iteration of inner loop takes 40ns
    } // including increment and termination checks
    return ret;
}

/** write a bit to I2C bus, d is period in us */
void I2C_WriteBit(unsigned char c, int d)
{
    if (c > 0) set_I2C_SDA_hi();
    else      set_I2C_SDA_lo();

    set_I2C_SCL_hi();
    precise_delay(d);

    set_I2C_SCL_lo();
    precise_delay(d);

    if (c > 0) set_I2C_SDA_lo();
    precise_delay(d);
}

/** write byte to I2C bus, bit_period is period in us for each bit in byte sent. */
void I2C_Write (unsigned char x, int bit_period)
{
    for (char i = 0; i < 8; i++) {
        I2C_WriteBit(x & 128, bit_period);
        x <<= 1;
    }
}
```

END OF EXAMINATION

Appendix A: Verilog Cheat sheet

Numbers and constants

Example: 4-bit constant 10 in binary, hex and in decimal: 4'b1010 == 4'ha -- 4'd10

(numbers are unsigned by default)

Concatenation of bits using {}

4'b1011 == {2'b10, 2'b11}

Constants are declared using parameter:

parameter myparam = 51

Operators

Arithmetic: and (+), subtract (-), multiply (*), divide (/) and modulus (%) all provided.

Shift: left (<<), shift right (>>)

Relational ops: equal (==), not-equal (!=), less-than (<), less-than or equal (<=), greater-than (>), greater-than or equal (>=).

Bitwise ops: and (&), or (|), xor (^), not (~)

Logical operators: and (&&) or (||) not (!) note that these work as in C, e.g. (2 && 1) == 1

Bit reduction operators: [n] n=bit to extract

Conditional operator: ? to multiplex result

Example: (a==1)? funcif1 : funcif0

The above is equivalent to:

```
((a==1) && funcif1)
|| ((a!=1) && funcif0)
```

Registers and wires

Declaring a 4 bit wire with index starting at 0:

```
wire [3:0] w;
```

Declaring an 8 bit register:

```
reg [7:0] r;
```

Declaring a 32 element memory 8 bits wide:

```
reg [7:0] mem [0:31]
```

Bit extract example:

r[5:2] returns 4 bits between pos 2 to 5 inclusive

Assignment

Assignment to wires uses the assign primitive outside an always block, e.g.:

```
assign mywire = a & b
```

Registers are assigned to inside an always block which specifies where the clock comes from, e.g.:

```
always@(posedge myclock)
```

```
cnt = cnt + 1;
```

Blocking vs. unblocking assignment <= vs. =

The <= assignment operator is non-blocking (i.e. if use in an always@(posedge) it will be performed on every positive edge. If you have many non-blocking assignments they will all be updated in parallel. The <= operator must be used inside an always block – you can't use it in an assign statement.

The blocking assignment operator = can be used in either an assign block or an always block. But it causes assignments to be performed in sequential order. This tends to result in slower circuits, so avoid using it (especially for synthesized circuits) unless you have to.

Case and if statements

Case and if statements are used inside an always block to conditionally update state. e.g.:

```
always @(posedge clock)
  if (add1 && add2) r <= r+3;
  else if (add2) r <= r+2;
  else if (add1) r <= r+1;
```

Note that we don't need to specify what happens when add1 and add2 are both false since the default behavior is that r will not be updated. Equivalent function using a case statement:

```
always @(posedge clock)
  case({add2,add1})
    2'b11 : r <= r+3;
    2'b10 : r <= r+2;
    2'b01 : r <= r+1;
    default: r <= r;
  endcase
```

Module declarations

Modules pass inputs, outputs as wires by default.

```
module ModName (
  output reg [3:0] result, // register output
  input [1:0] bitsin, input clk, inout bidirectnl );
  ... code ...
endmodule
```

Verilog Simulation / ISIM commands

```
$display ("a string to display");
$monitor ("like printf. Vals: %d %b", decv,bitv);
#100 // wait 100ns or simulation moments
$finish // end simulation
```

Appendix B: ARM Assembly Language Cheatsheet

Memory access instructions

```
LDR Rd, [Rn] ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off] ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn] ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off] ; store 32-bit Rt to [Rn+off]
PUSH {Rt} ; push 32-bit Rt onto stack
POP {Rd} ; pop 32-bit number from stack into Rd

MOV{S} Rd, <op2> ; set Rd equal to op2
MOV Rd, #iml6 ; set Rd equal to iml6, iml6 is 0 to 65535
MVN{S} Rd, <op2> ; set Rd equal to -op2
```

Branch instructions

```
B label ; branch to label Always
BEQ label ; branch if Z == 1 Equal
BNE label ; branch if Z == 0 Not equal
BCS label ; branch if C == 1 Higher or same, unsigned ≥
BHS label ; branch if C == 1 Higher or same, unsigned ≥
BCC label ; branch if C == 0 Lower, unsigned <
BLO label ; branch if C == 0 Lower, unsigned <
BMI label ; branch if N == 1 Negative
BPL label ; branch if N == 0 Positive or zero
BVS label ; branch if V == 1 Overflow
BVC label ; branch if V == 0 No overflow
BHI label ; branch if C==1 and Z==0 Higher, unsigned >
BLS label ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label ; branch if N == V Greater than or equal, signed ≥
BLT label ; branch if N != V Less than, signed <
BGT label ; branch if Z==0 and N==V Greater than, signed >
BLE label ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm ; branch indirect to location specified by Rm
BL label ; branch to subroutine at label
BLX Rm ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, <op2> ; Rd=Rn&op2 (op2 is 32 bits)
ORR{S} {Rd}, Rn, <op2> ; Rd=Rn|op2 (op2 is 32 bits)
EOR{S} {Rd}, Rn, <op2> ; Rd=Rn^op2 (op2 is 32 bits)
BIC{S} {Rd}, Rn, <op2> ; Rd=Rn&(~op2) (op2 is 32 bits)
ORN{S} {Rd}, Rn, <op2> ; Rd=Rn|(~op2) (op2 is 32 bits)
LSR{S} Rd, Rm, Rs ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n ; logical shift right Rd=Rm>>n (unsigned)
ASR{S} Rd, Rm, Rs ; arithmetic shift right Rd=Rm>>Rs (signed)
ASR{S} Rd, Rm, #n ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, <op2> ; Rd = Rn + op2
ADD{S} {Rd}, Rn, #iml2 ; Rd = Rn + iml2, iml2 is 0 to 4095
SUB{S} {Rd}, Rn, <op2> ; Rd = Rn - op2
SUB{S} {Rd}, Rn, #iml2 ; Rd = Rn - iml2, iml2 is 0 to 4095
RSB{S} {Rd}, Rn, <op2> ; Rd = op2 - Rn
RSB{S} {Rd}, Rn, #iml2 ; Rd = iml2 - Rn
CMP Rn, <op2> ; Rn - op2 sets the NZVC bits
MUL{S} {Rd}, Rn, Rm ; Rd = Rn * Rm signed or unsigned
```

Notes

Ra Rd Rm Rn Rt represent 32-bit registers

value any 32-bit value: signed, unsigned, or address

{S} if S is present, instruction will set condition codes

#im12 any value from 0 to 4095

#im16 any value from 0 to 65535

{Rd,} if Rd is present Rd is destination, otherwise Rn

#n any value from 0 to 31

#off any value from -255 to 4095

label any address within the ROM of the microcontroller

op2 the value generated by <op2>